VORTEXA 3.0 | REPORT SUBMISSION

RailFlow

Railway Disruption & Platform Conflict Simulator

Problem Understanding · Solution Proposal · System Architecture

"The planner may be wrong; the validator cannot be fooled; the AI never touches a platform." This is RailFlow's core design rule. AI turns messy radio and text reports into structured, checkable proposals. Optimisation proposes platform plans. An independent rule checker, not the AI and not the optimiser, decides whether a plan is safe to apply.

Submission details

Field	Details
Hackathon	Vortexa 3.0 — 12 Hour Hackathon
Host institute	Dr. D. Y. Patil Institute of Technology, Pimpri, Pune
Team ID (Team Code)	**VTX26-92646**
Team Name	**Runtime Renegades**
Problem Statement ID	**P-03**
Problem Statement title	RailFlow: Railway Disruption & Platform Conflict Simulator
Domain	Domain 6 — Public Systems & Civic Technology
Submission date	11 October 2026

Team members

#	Name	Role	College · Programme	Email
1	**Gurjas Singh Gandhi**	Team Leader	P.E.S. Modern College of Engineering, Pune · MCA	gurjasgandhi76@gmail.com
2	**Aaditi Choramle**	Member	P.E.S. Modern College of Engineering, Pune · MCA	aaditichoramle14@gmail.com

Contents

1. Executive summary
2. Problem understanding
3. Proposed solution
4. Intended users
5. AI integration
6. Differentiation from existing solutions
7. System architecture
8. Technology stack
9. Validation, testing and measured results
10. The product in screenshots
11. Limitations, roadmap and conclusion

How to read this report in five minutes: read §1 (the summary), look at Figure 1 (architecture) and Figure 2 (the flow from disruption to applied plan), then read the comparison table in §6.

1. Executive summary

A busy junction runs dozens of trains through a handful of platforms. When one train runs late or a platform is closed for inspection, the controller has minutes to re-plan: which trains move, which wait, and is the new plan actually conflict-free? Today this depends on paper charts, phone calls and experience. Mistakes, such as two trains booked for one platform or a long train sent to a short platform, are only discovered when it is too late to fix them.

RailFlow is a working, offline decision-support simulator that a section controller can use live:

- **Understands disruptions in plain language.** "11209 running 30 min late, PF 3 shut 14:00–15:30" is typed or spoken, and becomes structured, checkable proposals. English, Hinglish and Devanagari Hindi/Marathi are supported, through a local LLM with a deterministic fallback.
- **Re-plans only what must change.** An OR-Tools CP-SAT optimiser moves only the affected trains and offers trade-offs: *A · minimum delay* versus *B · fewest platform changes*, with a greedy baseline for comparison.
- **Proves every plan before it is used.** An independent validator (rules V1–V10) checks every plan with a sweep line and lists each conflict by train, platform and minutes. Only this validator can show the VALID badge.
- **Explains and records every decision.** Each moved train has a reason card and a *caused-by* chain. Every action, including refused ones, goes into a tamper-evident SHA-256 audit chain.

Headline result	Value (measured, committed in the repo)
Plans validated across 20 random peak-hour disruption scenarios	60 / 60 VALID (CP-SAT × 2 + greedy)
Weighted delay: CP-SAT min-delay vs greedy	1526 vs 1847 (**−17 %**)
Platform changes: CP-SAT min-changes vs greedy	25 vs 92 (**−73 %**)
Optimality	20 / 20 proven OPTIMAL, median solve 66 ms
AI parser exact-match accuracy (52 labelled messages)	LLM 44/52 vs regex 36/52; paraphrases 8/8 vs 3/8
Wrong AI values that reached the operator as "ready"	**0** (9 / 9 caught by the deterministic check)
Automated tests	130 backend tests incl. 2 × 200 property-based cases, all passing

2. Problem understanding

2.1 The operational reality

At a junction station each platform can host only one train at a time, and it must stay empty for a minimum gap after a train leaves (for safety and for passengers to clear). Many trains also have hard constraints:

- a 24-coach express cannot use a 16-coach platform, and a goods train needs a loop long enough for its wagons;
- some services must use a particular platform, for example for a connecting line or a parcel facility;
- a train already standing at a platform can only be delayed in its departure; it cannot "arrive later";
- a train cannot depart before its scheduled time, and its dwell (time at the platform) must be preserved.

A single disruption cascades. When train A is held, it occupies its platform longer, so train B must wait or move, which then displaces train C. The controller needs to see this chain, not just the end result.

2.2 Why current practice fails

Pain point	Consequence
Platform charts maintained by hand or in spreadsheets	Gap violations and overlaps are invisible until a train is at the signal
Disruption reports arrive as free-text radio or phone messages	Re-typing is slow and error-prone; numbers get misheard ("thirty" vs "thirteen")
Re-planning is done by hand under time pressure	Too many trains are moved, or delay is not minimised; trade-offs are never compared
No independent check of the new plan	One wrong assumption propagates to many trains
Generic AI chatbots	Can "hallucinate" a train number or a platform and give no proof the plan is safe
No record of who decided what	Post-incident review is guesswork

2.3 What problem statement P-03 asks for

#	P-03 requirement	How RailFlow meets it
1	Conflict-free platform assignment with a minimum gap, honouring required platforms	CP-SAT planner with interval constraints; occupancy = [arr, dep + min_gap)
2	Detect conflicts and report them by train and platform	Validator V8 reports each pair with overlap, clearance shortfall and reserved-time minutes
3	Re-plan after delays and closures, with reasons recorded	Two-stage affected-set replanning; ReasonCard + caused_by chain per train
4	Validate every revised timetable independently	validator.py imports only the models and the standard library; property tests inject faults
5	AI converts free text into disruptions and explains trade-offs without deciding assignments	AI outputs only Disruption[] or text; schema makes a platform assignment impossible to express
Rules	Local fallback; fictional data; reproducible live testing	Runs fully offline (Ollama local + regex/template fallback); fictional Shivneri Junction; scenarios S1–S8 seeded

3. Proposed solution

RailFlow separates **understanding** (AI), **deciding** (optimisation) and **proving** (validation), and puts a human at each point where a decision has consequences.

3.1 Core capabilities

① Detect

Load a timetable (.xlsx/.csv/.json) and press **Check as scheduled**.
Every conflict is listed with minutes and drawn on the Gantt chart. The demo workbook's 3 planted conflicts are reproduced exactly.

② Assign

Assign platforms produces a conflict-free plan for the whole day, respecting length, required platforms and the minimum gap.
Result: VALID · FULLY ASSIGNED.

③ Understand

Type or record a report. The local LLM proposes delays and closures, each linked to the words it came from. Unknown trains, impossible times and numbers not found in the message are flagged.

④ Re-plan

Only the affected set moves. Revisions A (min delay), B (fewest changes) and a greedy baseline are compared side by side on hold, weighted delay, changes and unassigned trains.

⑤ Prove

The independent validator checks all ten rules (V1–V10). A planner bug injected on purpose (S5) is caught: NOT VALID, and Apply is disabled.

⑥ Explain & apply

A grounded explanation of the trade-off; reason card per train; Apply = hash check + re-validation + one transaction, recorded in the audit chain.

3.2 Reproducible demo scenarios

Scenario	Situation	Expected (and observed) outcome
S1	Normal day	VALID · FULLY ASSIGNED · 0 changes
S2	Radio report: 11209 +30 min, PF 3 closed 14:00–15:30	Affected 3 of 30 · 3 moves · hold 16 · all revisions agree
S2-late	Same report at 14:05	Delay rejected: 11209 is already at PF 3
S3	Spreadsheet with planted conflicts	GAP P1 2 min · GAP P2 3 min · OVERLAP P4 5 / reserved 10
S4	Report names a train not in the timetable	Rejected, never guessed
S5	S8 + deliberately injected planner fault	OVERLAP P1 12222↔51215 (7 min) → NOT VALID, Apply disabled
S6	Peak hour, two platforms closed	Goods train UNASSIGNED with reason; plan still VALID (PARTIAL)
S7	Ollama (AI) switched off	parser: regex · explainer: template · same plan as S2
S8	Departure delay at a platform + closure	A: 2 moves / weighted 76 vs B: 1 move / weighted 136; chain 51218 → 51215

3.3 Optimisation model in brief

Each train *t* chooses a platform *p* and a hold $h \geq 0$. On a platform, a train occupies the interval **[arr + h, dep + h + min_gap)** and a closure occupies **[start, end)**; intervals on one platform must not overlap (CP-SAT *NoOverlap*). Trains outside the affected set are frozen. The objective is **lexicographic**: first minimise Σ priority × unassigned trains; then the primary goal (weighted delay for A, platform changes for B); then the secondary goal. If the first, narrow affected set is infeasible, the solver widens it once and records that it did.

4. Intended users

Section / station controller (primary)

Receives radio and phone reports and re-plans platforms under time pressure.
Needs: fast and correct re-planning, visible trade-offs, the ability to override.
RailFlow role: controller. Can parse, plan, compare and apply.

Station master / supervisor

Monitors the station picture on a phone or tablet on the platform.
Needs: read-only live view, 2D/3D station view, per-train reasons.
RailFlow role: viewer. Read only; any write attempt is refused (403) and audited.

Operations admin / auditor

Manages accounts and reviews system health and decisions.
Needs: user management, tamper-evident audit trail, health dashboard.
RailFlow role: admin. Cannot plan or apply, so duties stay separated.

Trainees & planners

Learn disruption handling on realistic, fictional scenarios.
Needs: safe sandbox, replayable scenarios, explanations.
RailFlow: S1–S8 scenarios, fault injection and before/after views.

Accessibility and reach: the interface works on phones (360 px), tablets and desktops, in English, Hindi (हिंदी) and Marathi (मराठी). IDs, times and minutes are never translated, so train numbers read the same in every language.

5. AI integration

Under VORTEXA's common rules, AI must do real work (language understanding and extraction) but must not silently override decisions that have consequences. RailFlow uses AI exactly where language is the bottleneck and keeps it away from decisions.

AI does	AI never does
Reads free-text / Hinglish / Devanagari reports and proposes Delay and Closure events	Assign, move or hold a train; its output schema cannot express that
Highlights which words each proposal came from (source spans)	Invent a train or platform: unknown IDs are rejected, never guessed
Explains the trade-off between revisions in plain English	Decide whether a plan is valid; only validator.py shows that badge
Transcribes radio clips (optional faster-whisper, local)	Call any cloud service: everything runs on the laptop

5.1 The AI pipeline and its safety nets

1. **Local LLM parser.** Ollama `qwen2.5:3b-instruct` at temperature 0 returns JSON `Disruption[]` with a source span for each.
2. **Regex fallback.** If Ollama is down or returns unusable output, a deterministic parser takes over. The UI chip shows `parser: regex`.
3. **Deterministic proposal validator.** It checks that IDs exist, minutes are in range, times are ordered, the train has not departed or is not already at a platform, and that **every number in the proposal appears in the operator's own words**. If not, the proposal is shown as *needs clarification*, never *ready*.
4. **Human confirmation.** The controller edits, removes or confirms. *Confirm & plan* stays disabled until every proposal is valid.
5. **Grounded explainer.** Every number in the LLM's explanation must match a computed metric; otherwise a template explanation is used. Hindi and Marathi always use grounded templates.

5.2 Measured AI quality

Metric (52 labelled messages)	Regex	Local LLM
Exact-match messages	36 / 52	**44 / 52**
Event precision / recall	1.00 / 0.65	0.82 / 0.80
Paraphrases (“half an hour behind”, “engineering block”)	3 / 8	**8 / 8**
Devanagari messages	1 / 10	**5 / 10**
Unknown IDs correctly rejected	5 / 6	**6 / 6**
Wrong LLM values flagged before the operator	—	**9 / 9**
Median latency (CPU only)	0 ms	3.6 s

Why the combination works: the LLM's recall on real-world phrasing is far better than regex, and its occasional wrong numbers are always caught by the deterministic check. The operator gets the AI's understanding and none of its risk.

6. Differentiation from existing solutions

Capability	Manual charts / spreadsheets	Generic schedulers / solvers	LLM chatbots	RailFlow
Understands free-text / radio reports	✗	✗	✓	**✓ local, multilingual, with fallback**
Optimal re-planning with trade-offs	✗	✓ (one objective)	✗	**✓ A vs B vs baseline, lexicographic**
Moves only affected trains	Depends on person	Rarely	✗	**✓ two-stage affected set**
Independent proof that the plan is conflict-free	✗	Trusts its own output	✗	**✓ separate validator V1–V10**
Conflicts reported with minutes	✗	Partial	✗	**✓ per train, per platform**
Cannot hallucinate assignments	n/a	n/a	✗	**✓ by schema**
Explains why each train moved	✗	✗	Unverified text	**✓ grounded + caused_by chain**
Tamper-evident audit	✗	✗	✗	**✓ SHA-256 hash chain**
Works offline, no paid API	✓	Varies	✗ (usually cloud)	**✓ single laptop**
Phone / tablet, EN · हिं · मर	✗	✗	Partial	✓

What makes RailFlow unique

- **Separation of powers.** AI understands, the solver proposes, an independent validator decides, a human applies. Each can fail without causing harm, and S5 demonstrates a planner fault being caught live.
- **“What is applied is exactly what was validated.”** The revision is hashed at validation time. Apply re-hashes, re-validates and writes in one transaction. Stale or edited plans get HTTP 409.
- **Honest under failure.** With the AI switched off (S7), the system produces the same plans via regex and templates. The UI always shows which path produced each result.
- **Explainable cascades.** “51215 held 15 min because 51218’s departure delay occupied PF 2” is shown as a chain, not a guess.

7. System architecture

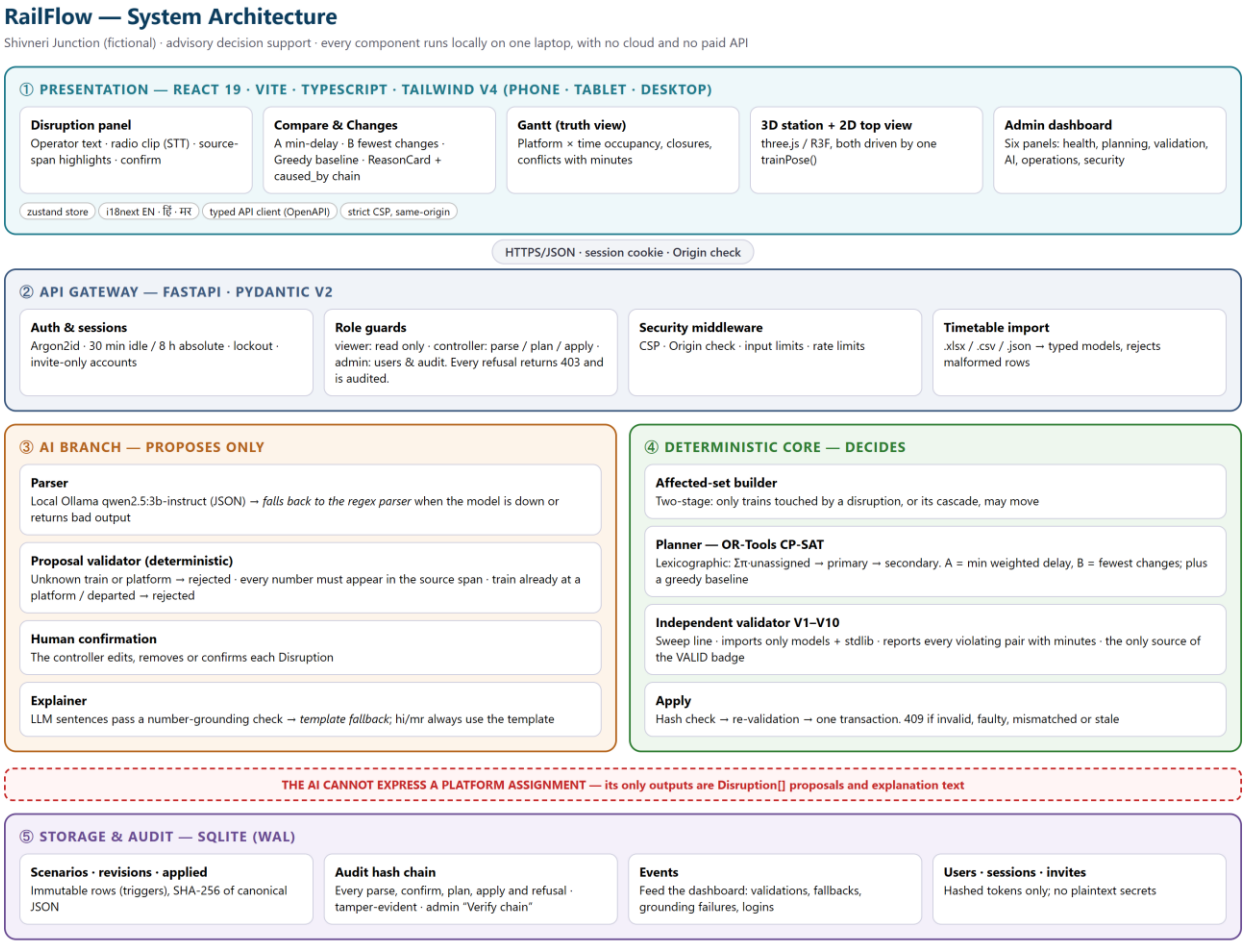


Figure 1. *RailFlow system architecture: five layers, with a hard wall between the AI branch (proposes) and the deterministic core (decides).*

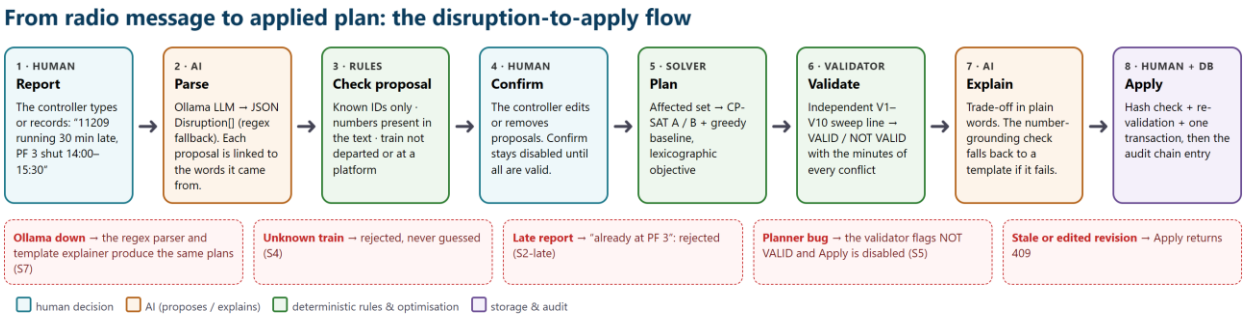


Figure 2. *Disruption-to-apply flow, with the failure paths that the system handles safely.*

7.1 Components

Layer	Component	Responsibility
Presentation	React 19 SPA	Disruption, Compare, Changes, ReasonCard, Gantt, 3D/2D station, admin dashboard; zustand store; i18next
API	FastAPI + Pydantic v2	Typed REST routes; role guards (viewer / controller / admin); refusal → 403 + audit
Security	auth.py, security.py	Argon2id passwords, server sessions (30 min idle / 8 h), lockout, invites, CSP, Origin check

Layer	Component	Responsibility
AI	parser_llm.py · parser_regex.py · proposals.py · explainer.py	LLM/regex parsing, deterministic proposal checks, grounded explanations
Core	state.py · planner_cpsat.py · planner_greedy.py	Affected-set construction, CP-SAT lexicographic optimisation, greedy baseline
Proof	validator.py	Independent sweep-line validator V1–V10; imports only models + standard library
Storage	db.py · schema.sql · audit.py · events.py	SQLite WAL, immutable rows via triggers, SHA-256 audit chain, dashboard events

7.2 Validator rules

Rule	Checks
V1	Every train present exactly once; no unknown trains or platforms
V2 – V3	Required / allowed platforms respected; platform length fits the train (coaches, wagons)
V4 – V5	Dwell preserved; times well-formed; no departure earlier than possible
V6 – V7	Holds within the allowed maximum; frozen (departed / at-platform) trains unchanged
V8	No overlap or minimum-gap violation; reports stay overlap, clearance shortfall and reserved-time minutes
V9	No train on a closed platform
V10	Every unassigned train has a recorded reason; capacity respected

7.3 Data, security and integrity

- **Data:** fictional Shivneri Junction (SVNJ), 5 platforms plus a goods loop, 30-train normal and conflict timetables and a 36-train peak timetable. No real personal or operational data.
- **Integrity:** each revision is stored with the SHA-256 of its canonical assignments, rows are immutable (triggers), and every action extends a hash chain that the admin can verify.
- **Security:** invite-only accounts, Argon2id, server-side sessions, role separation (the admin cannot plan or apply), a strict same-origin CSP, and an Origin check on writes.
- **Deployment:** one command (`start.ps1` / `start.sh`) builds the web app and serves the app and API from a single process on the laptop. Phones join over a local hotspot.

8. Technology stack

Area	Technology	Why
Backend	Python 3.12, FastAPI, Pydantic v2	Typed, fast, auto-generated OpenAPI → typed frontend client
Optimisation	Google OR-Tools CP-SAT	Exact interval scheduling; proves optimality in milliseconds at station scale
AI	Ollama + qwen2.5:3b-instruct (local), optional faster-whisper	Runs on a CPU laptop, offline, no paid API
Storage	SQLite (WAL), sqlite3	Zero-ops, transactional, portable
Security	argon2-cffi, CSP, sessions	Industry-standard password hashing and browser hardening
Frontend	React 19, Vite, TypeScript (strict), Tailwind v4, zustand	Fast, typed, responsive UI

Area	Technology	Why
Visualisation	three.js / React-Three-Fiber / drei, Recharts	3D station, top view, dashboard charts
i18n	i18next (EN · हिं · मर)	Regional language reach
Testing	pytest, Hypothesis, Vitest	Unit, API, property-based and reference-parity tests

9. Validation, testing and measured results

- **130 backend tests, all passing.** They cover the validator, validator independence, delay accounting, objective hierarchy and capacity, timetable import, parser, explainer grounding, API, auth, storage integrity, locales and STT.
- **Property-based testing (Hypothesis):** 200 random disruption cases check that every revision validates, and another 200 inject a planner fault to check that every fault is caught.
- **Reference parity:** results for S1, S2, S3, S5 and S8 match an independently written reference model (oracle) number for number.
- **Benchmark:** 20 seeded peak-hour scenarios (`results/bench.md`). **Parser evaluation:** 52 labelled messages (`results/parser_eval.md`).
- **End-to-end:** the full demo path was driven in a headless browser at 1366 px, 768 px and 360 px. Role refusals were checked (viewer → Apply 403, admin → Plan 403).

Method (20 scenarios, 36 trains)	Valid	Weighted delay	Platform changes	Unassigned	Median solve
Greedy baseline	20/20	1847	92	5	0 ms
CP-SAT A · min-delay	20/20	**1526**	91	4	66 ms
CP-SAT B · min-changes	20/20	4008	**25**	4	66 ms

10. The product in screenshots

All screenshots come from the running system (`app_screenshots_&_screen_recordings/`, which also contains the full screen recording `railflow_full_demo.mp4`).

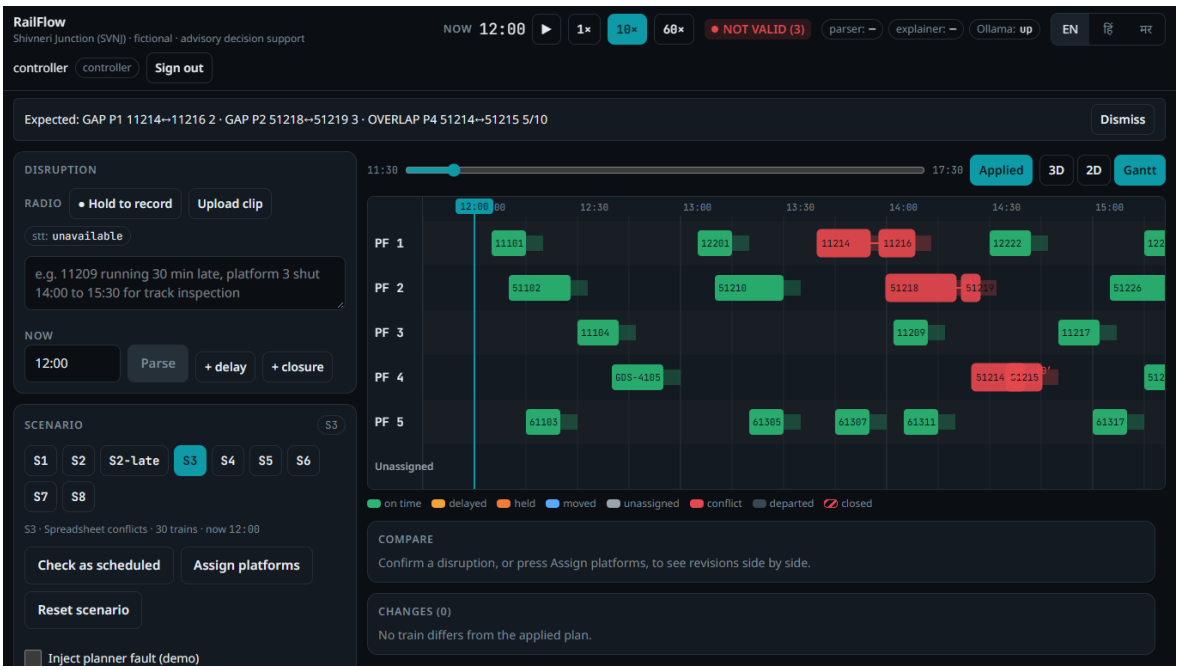


Figure 3. *S3: the three planted conflicts found as scheduled, each with minutes, shown on the Gantt chart.*

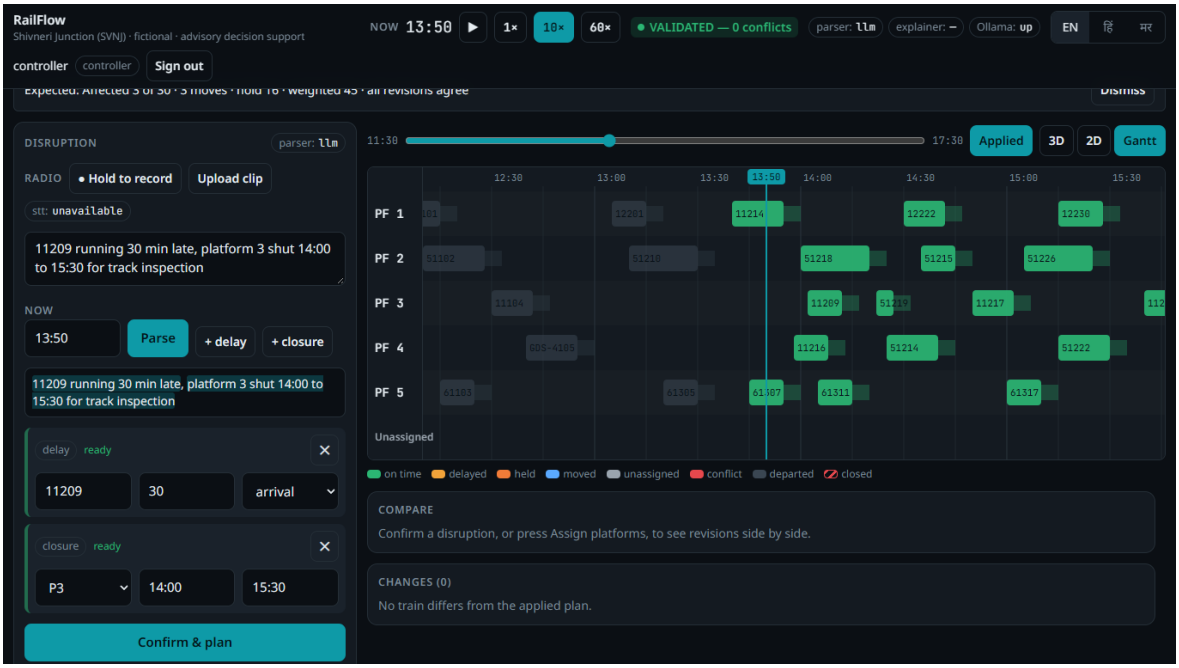


Figure 4. *S2: the local LLM turns the radio report into proposals linked to the words they came from; the controller confirms.*

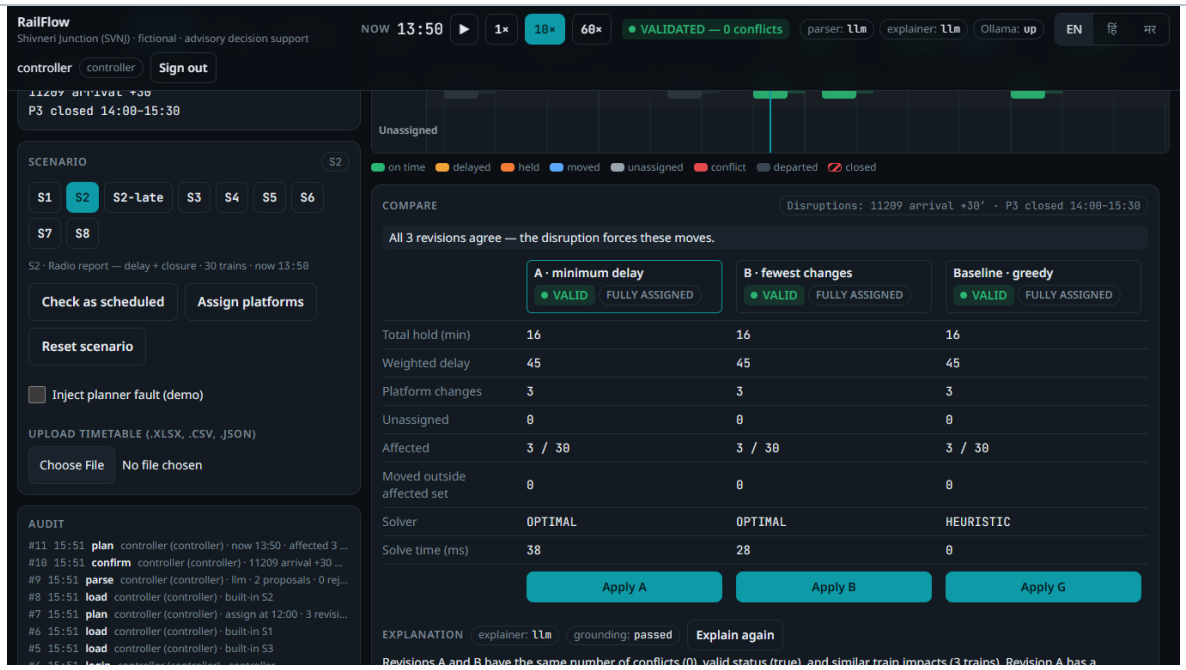


Figure 5. *S2: revisions A, B and the greedy baseline compared. The independent validator marks each VALID, and the explanation passed grounding.*

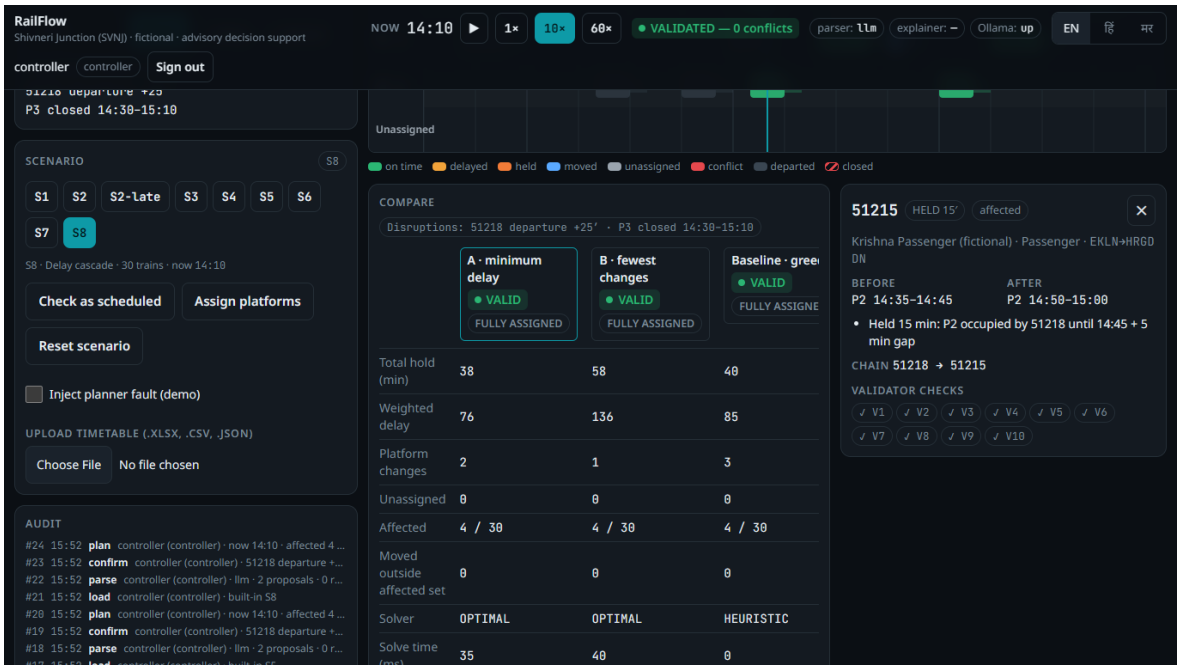


Figure 6. *S8: trade-off between minimum delay and fewest changes; the reason card shows the caused-by chain.*

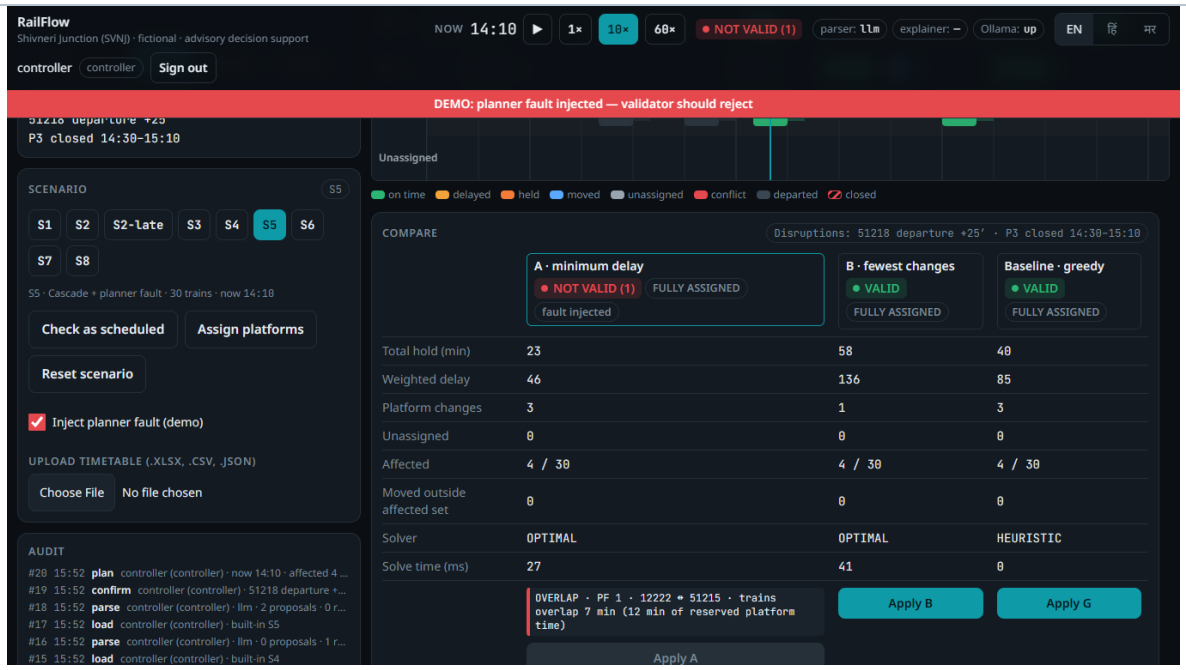


Figure 7. *S5: a deliberately injected planner fault is caught. NOT VALID with minutes, and Apply is disabled.*

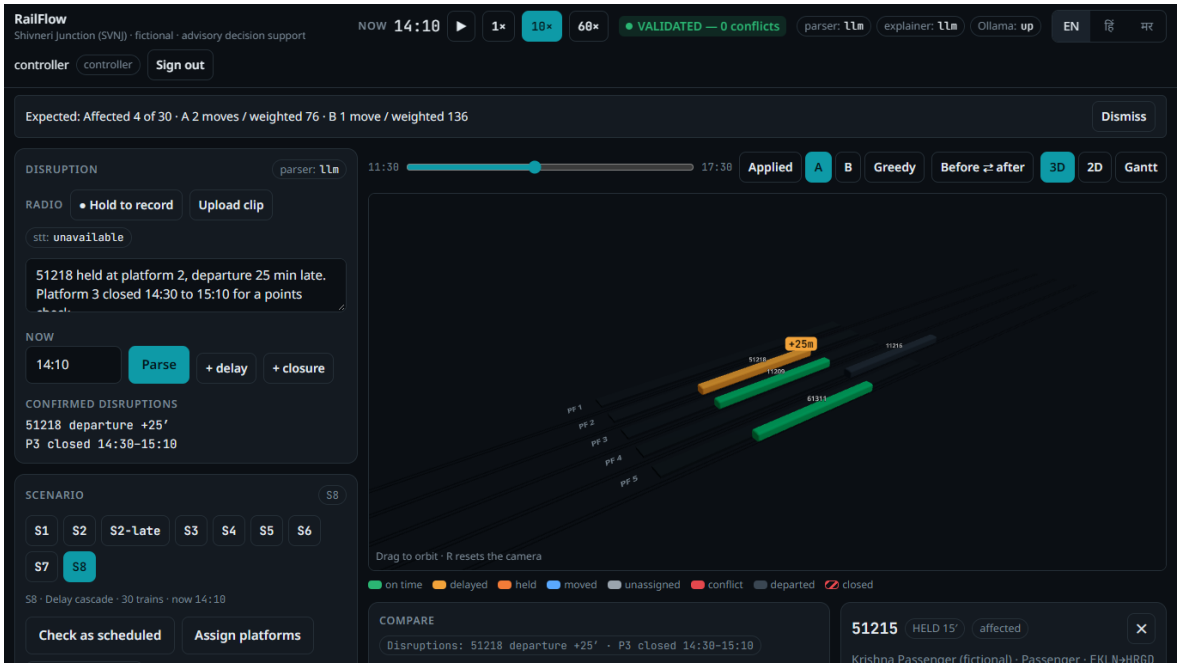


Figure 8. *3D station view driven by the same train positions as the Gantt chart.*

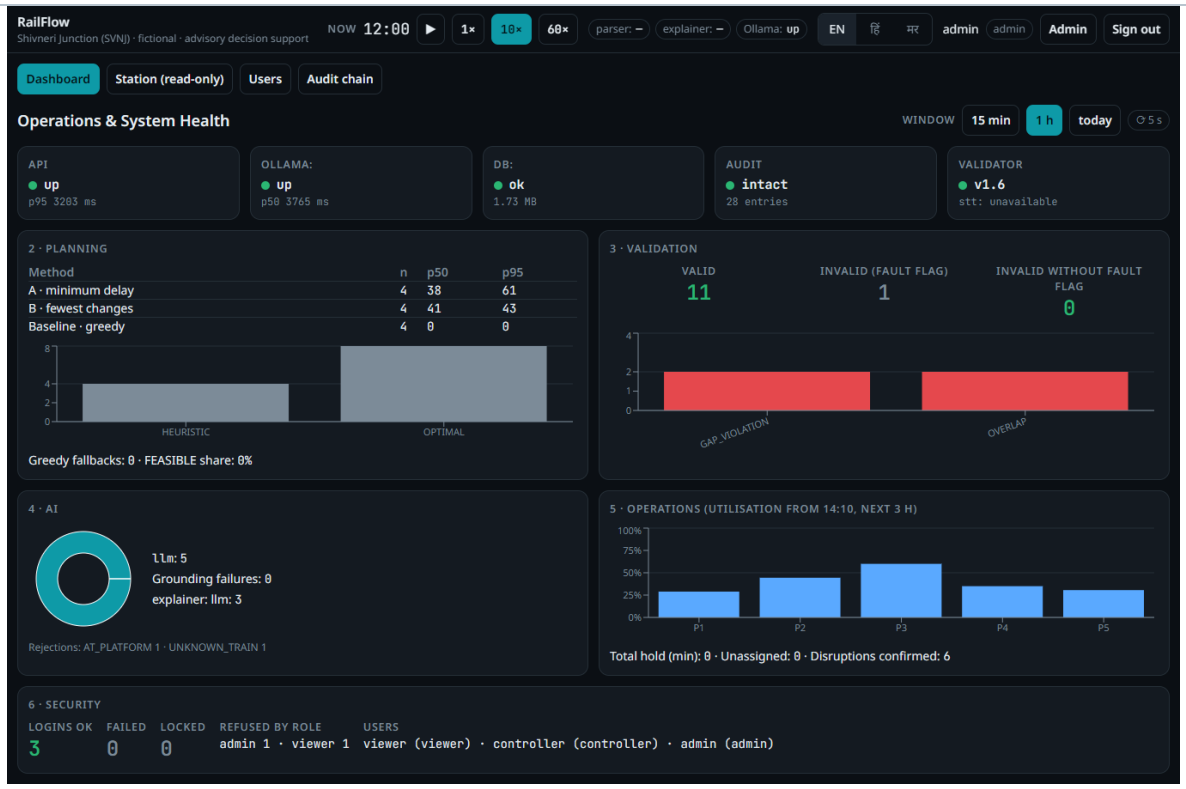


Figure 9. *Admin dashboard: health, planning, validation, AI, operations and security panels.*

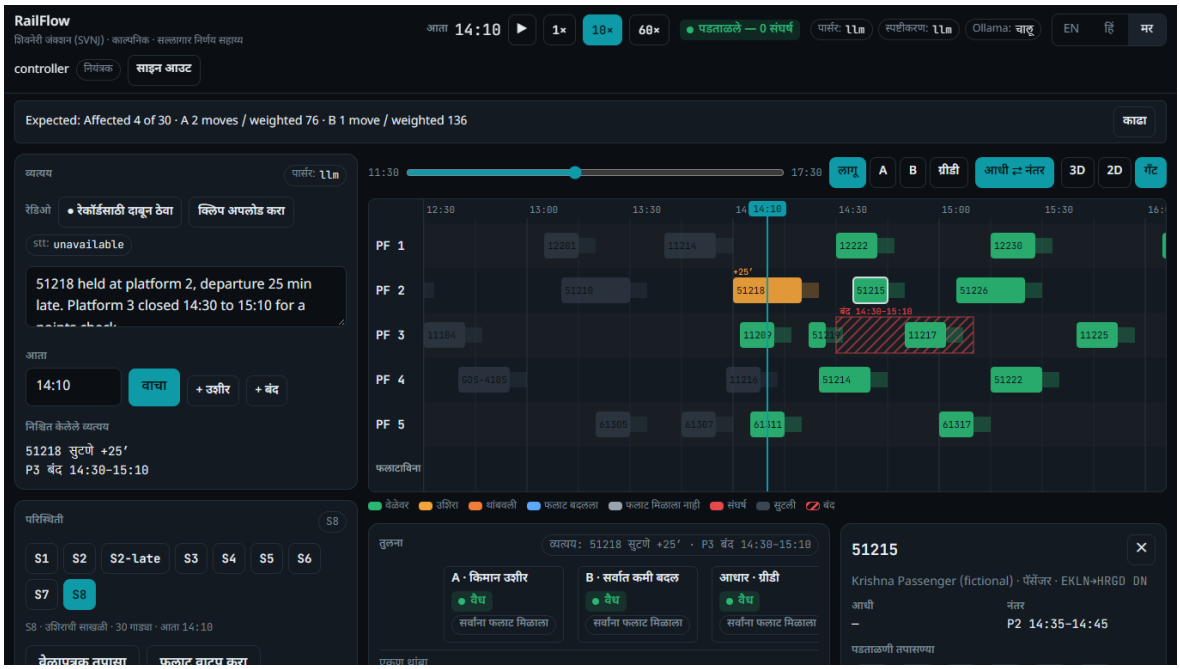


Figure 10. *The same screen in Marathi. IDs, times and minutes are never translated.*

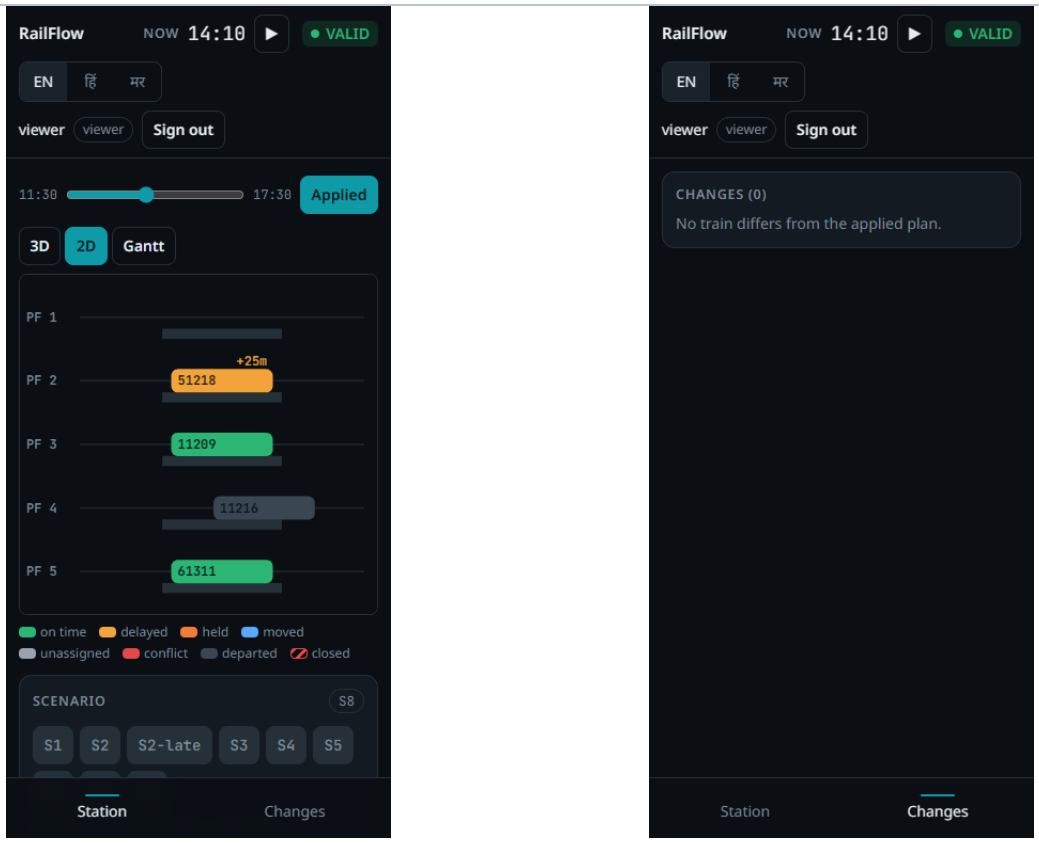


Figure 11. *Phone layout (360 px) for a station-master viewer: read-only station view and changes list.*

11. Limitations, roadmap and conclusion

11.1 Honest limitations

- Single station. Route conflicts in the station throat (V11) are designed and measured, but not yet built.
- Dwell time is preserved, never compressed. There are no passenger-connection or crew constraints yet.
- Hindi and Marathi strings need review by a native speaker. Devanagari number words are always sent for clarification.
- Speech-to-text is built and tested with a mocked model; it still has to be checked against the real clips on the demo laptop.
- Advisory decision support on fictional data: not a safety system and not connected to signalling.

11.2 Roadmap

Next step	Value
V11 route conflicts in the station throat	Catch conflicting movements across points and crossovers, not just platforms
Multi-station / section view	Propagate delays along a corridor
Passenger connections and crew links	Protect transfers; avoid crew rule breaches
Integration adapters (read-only)	Import working timetables and live train positions from existing systems
Field pilot with controllers	Native-language review, usability study, offline dry runs

11.3 Conclusion

RailFlow shows that AI can be useful in a safety-relevant setting without being trusted blindly. The language model does what it is good at, turning messy human reports into structured proposals. An optimiser finds the best trade-offs, and an independent validator proves every plan before a human applies it, with every step on record. The result is faster, explainable and verifiable disruption handling that runs on a single laptop, works offline and speaks the controller's language.

Repository: https://github.com/Gurjas2112/RailFlow (main branch) · **Run:** `./start.ps1` → http://127.0.0.1:8000 · **Evidence:** `results/bench.md`, `results/parser_eval.md`, `app_screenshots_&_screen_recordings/`